

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ «ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Отчёт по домашней работе

«Разработка алгоритма подсчёта инверсий и значительно инвертированных пар в массиве»

Студент: Ошаров Александр

Семинарист: Маров Руслан

Дата: 2025-10-12

1. Алгоритм CINV для подсчёта обычных инверсий

1.1. Постановка задачи

Необходимо разработать алгоритм $CINV$, который вычисляет количество **инверсий** в массиве $A = [a_1, a_2, \dots, a_n]$ — пар (i, j) таких, что $i < j$ и $a_i > a_j$. Алгоритм должен иметь временную сложность $O(n \log n)$.

1.2. Суть алгоритма: сортировка слиянием с подсчётом инверсий

Алгоритм основан на парадигме **разделяй и властвуй (Divide and Conquer)** и является модификацией алгоритма **сортировки слиянием**.

Шаг DIVIDE:

Массив A делится на две половины: левую $L = A[1.. \lfloor n/2 \rfloor]$ и правую $R = A[\lfloor n/2 \rfloor + 1.. n]$.

Шаг CONQUER:

Рекурсивно вычисляются инверсии в левой половине (inv_L) и в правой половине (inv_R).

Шаг COMBINE:

Вычисляются **перекрёстные инверсии** — пары (i, j) , где i находится в левой половине, а j — в правой, и при этом $a_i > a_j$. Это делается во время слияния двух отсортированных

половин.

Ключевая идея: При слиянии, если элемент из левой половины больше текущего элемента из правой половины, то он больше **всех оставшихся** элементов в правой половине (поскольку обе половины уже отсортированы). Поэтому к счётчику добавляется количество оставшихся элементов в правой половине.

1.3. Рекуррентное соотношение

Пусть $T(n)$ — временная сложность алгоритма для массива размера n .

- **DIVIDE:** деление на две части — $O(1)$.
- **CONQUER:** два рекурсивных вызова — $2T(n/2)$.
- **COMBINE:** слияние и подсчёт перекрёстных инверсий — $O(n)$.

Функция `mergeAndCount` выполняет один проход по двум половинам, каждый элемент обрабатывается ровно один раз. Поэтому её временная сложность — $O(n)$.

Следовательно:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n).$$

Применяя **основную теорему**, получаем:

- $a = 2, b = 2, f(n) = O(n)$,
- $n^{\log_b a} = n^{\log_2 2} = n^1 = n$,
- $f(n) = \Theta(n) \rightarrow$ случай 2.

Следовательно, $T(n) = \Theta(n \log n)$.

Вывод: Алгоритм удовлетворяет требованию $O(n \log n)$.

1.4. Возвращает ли алгоритм минимальное количество инверсий?

Да. Количество инверсий — это **фиксированная характеристика массива**, определяемая как число пар (i, j) с $i < j$ и $a_i > a_j$. Алгоритм **не пытается минимизировать** количество инверсий — он просто **подсчитывает их точное количество**. Это значение является минимальным в том смысле, что оно не зависит от способа подсчёта — оно однозначно определено структурой массива.

2. Модификация алгоритма для подсчёта значительно инвертированных пар

2.1. Постановка задачи

Теперь необходимо подсчитать количество **значительно инвертированных пар** — пар (i, j) , где $i < j$ и $a_i > 2a_j$.

2.2. Изменения в алгоритме

Основная структура алгоритма остаётся той же: **разделение, рекурсия, слияние**. Изменения происходят только на шаге **COMBINE**.

Изменение в шаге COMBINE:

При слиянии двух отсортированных половин, вместо подсчёта всех пар, где элемент левой половины больше элемента правой, мы считаем пары, где элемент левой половины **больше удвоенного** элемента правой половины.

Важное замечание: Поскольку половины отсортированы, но условие $a_i > 2a_j$ **не монотонно** относительно a_j (если a_j растёт, $2a_j$ тоже растёт, поэтому условие $a_i > 2a_j$ становится **сложнее**), мы не можем использовать простое правило "если $a_i > a_j$, то $a_i > a_k$ для всех $k > j$ ".

Решение: Для каждого элемента a_i в левой половине, мы находим **количество элементов a_j в правой половине**, для которых $a_i > 2a_j$. Это можно сделать за $O(n)$ времени с помощью **двух указателей**, так как обе половины отсортированы.

Указатель j двигается только вперёд, и никогда не возвращается назад. Поэтому общее количество итераций цикла `while` не превышает длины правой половины. Следовательно, сложность — $O(n)$.

Алгоритм подсчёта перекрёстных значительно инвертированных пар:

1. Пусть L и R — отсортированные левая и правая половины.
2. Инициализируем указатель $j = 0$ для правой половины.
3. Для каждого элемента l в левой половине:
 - Пока $j < \text{len}(R)$ и $l > 2 * R[j]$, увеличиваем j .
 - Добавляем к счётчику j (количество элементов в правой половине, которые меньше $l/2$).

4. Общее количество перекрёстных значительно инвертированных пар — сумма по всем l .

Этот проход выполняется за $O(n)$, так как каждый указатель двигается только вперёд.

2.3. Асимптотическая сложность

Рекуррентное соотношение остаётся тем же:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n),$$

поскольку шаг COMBINE по-прежнему работает за $O(n)$. Следовательно, $T(n) = \Theta(n \log n)$.

Вывод: Асимптотическая верхняя граница не изменяется.

3. Реализация на C++

Ниже приведена реализация обоих алгоритмов на C++.

```
#include <windows.h>

#include <algorithm>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <vector>

using namespace std;

void SetConsoleToCP1251() {
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);
}

long long mergeAndCount(vector<int>& arr, int left, int mid, int right) {
    vector<int> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;
    long long inv_count = 0;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        }
```

```

    } else {
        temp[k++] = arr[j++];
        inv_count += (mid - i + 1);
    }
}

while (i <= mid) temp[k++] = arr[i++];
while (j <= right) temp[k++] = arr[j++];

for (int idx = 0; idx < k; ++idx) arr[left + idx] = temp[idx];

return inv_count;
}

long long countInversions(vector<int>& arr, int left, int right) {
    if (left >= right) return 0;
    int mid = left + (right - left) / 2;
    long long inv_count = 0;
    inv_count += countInversions(arr, left, mid);
    inv_count += countInversions(arr, mid + 1, right);
    inv_count += mergeAndCount(arr, left, mid, right);
    return inv_count;
}

long long countSignificantInversions(vector<int>& arr, int left, int right) {
    if (left >= right) return 0;
    int mid = left + (right - left) / 2;
    long long inv_count = 0;
    inv_count += countSignificantInversions(arr, left, mid);
    inv_count += countSignificantInversions(arr, mid + 1, right);

    vector<int> L(arr.begin() + left, arr.begin() + mid + 1);
    vector<int> R(arr.begin() + mid + 1, arr.begin() + right + 1);
    sort(L.begin(), L.end());
    sort(R.begin(), R.end());

    int j = 0;
    for (int x : L) {
        while (j < R.size() && static_cast<long long>(x) > 2LL * R[j]) j++;
        inv_count += j;
    }

    vector<int> temp;
    merge(L.begin(), L.end(), R.begin(), R.end(), back_inserter(temp));
    for (size_t idx = 0; idx < temp.size(); ++idx) arr[left + idx] = temp[idx];
}

```

```

    return inv_count;
}

long long countInversions(vector<int> arr) {
    return countInversions(arr, 0, static_cast<int>(arr.size()) - 1);
}

long long countSignificantInversions(vector<int> arr) {
    return countSignificantInversions(arr, 0, static_cast<int>(arr.size()) - 1);
}

string formatArray(const vector<int>& arr) {
    stringstream ss;
    ss << "[";
    for (size_t i = 0; i < arr.size(); ++i) {
        if (i > 0) ss << " ";
        ss << arr[i];
    }
    ss << "]";
    return ss.str();
}

void testAlgorithm(bool useRussian) {
    cout << (useRussian ? "=== Тестирование алгоритма ==="
        : "=== Algorithm Test ===")
        << "\n\n";

    vector<vector<int>> tests = {{1, 3, 4, 2, 5},
        {5, 3, 2, 4, 1},
        {1, 2, 3, 4, 5},
        {5, 4, 3, 2, 1},
        {10, 1, 8, 3, 6}};

    if (useRussian) {
        // Двухстрочный заголовок для лучшей читаемости
        cout << setw(14) << "Входной массив"
            << " | " << setw(10) << "Обычные"
            << " | " << setw(10) << "Полученные"
            << " | " << setw(16) << "Значимые"
            << " | " << setw(10) << "Полученные" << "\n";
        cout << setw(14) << ""
            << " | " << setw(10) << "инверсии"
            << " | " << setw(10) << "обычные"
            << " | " << setw(16) << "инверсии"
            << " | " << setw(10) << "значимые" << "\n";
        cout << setw(14) << ""

```

```

        << " | " << setw(10) << " "
        << " | " << setw(10) << "инверсии"
        << " | " << setw(16) << " "
        << " | " << setw(10) << "инверсии" << "\n";
    cout << string(78, '-') << "\n";
} else {
    cout << setw(14) << "Input"
        << " | " << setw(10) << "Inversions"
        << " | " << setw(10) << "Got (Inv)"
        << " | " << setw(16) << "Sig Inversions"
        << " | " << setw(10) << "Got (Sig)" << "\n";
    cout << string(78, '-') << "\n";
}

for (const auto& test : tests) {
    long long expected_inv = 0, expected_sig_inv = 0;
    for (size_t i = 0; i < test.size(); ++i) {
        for (size_t j = i + 1; j < test.size(); ++j) {
            if (test[i] > test[j]) expected_inv++;
            if (static_cast<long long>(test[i]) > 2LL * test[j])
expected_sig_inv++;
        }
    }

    long long got_inv = countInversions(test);
    long long got_sig_inv = countSignificantInversions(test);

    cout << setw(14) << formatArray(test) << " | " << setw(10) << expected_inv
        << " | " << setw(10) << got_inv << " | " << setw(16)
        << expected_sig_inv << " | " << setw(10) << got_sig_inv << "\n";
}
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    SetConsoleToCP1251();

    cout << "Выберите язык вывода:\n";
    cout << "1 – Русский\n";
    cout << "2 – English\n";
    cout << "Ваш выбор (1 или 2): ";

    int choice;
    cin >> choice;

```

```
bool useRussian = (choice == 1);

cout << "\n";
testAlgorithm(useRussian);

return 0;
}
```

Рассмотрим:

```
for (size_t i = 0; i < test.size(); ++i) {
    for (size_t j = i + 1; j < test.size(); ++j) {
        if (test[i] > test[j]) expected_inv++;
        if (static_cast<long long>(test[i]) > 2LL * test[j]) expected_sig_inv++;
    }
}
```

Ручной подсчёт выполняется за $O(n^2)$, что неэффективно для больших массивов, но здесь используется только для тестирования корректности алгоритма на небольших входах.

Примечание. Поскольку массив содержит уникальные значения, условие $a_i > a_j$ эквивалентно $a_i \geq a_j$, и нам не нужно рассматривать случай равенства.